

Drakens Energy 360

An end-to-end synthetic data engineering platform for a downstream energy business

Report generated 2026-09-07 · Warehouse: drakens360_test.duckdb

Independent synthetic portfolio project. Drakens Energy is a fictional company invented for this project. It does not exist. Every site, customer, employee, supplier, asset, coordinate, price, volume, margin and incident in this report is randomly generated. Site coordinates are South African town centroids plus random jitter and do not represent any real service station.

Summary

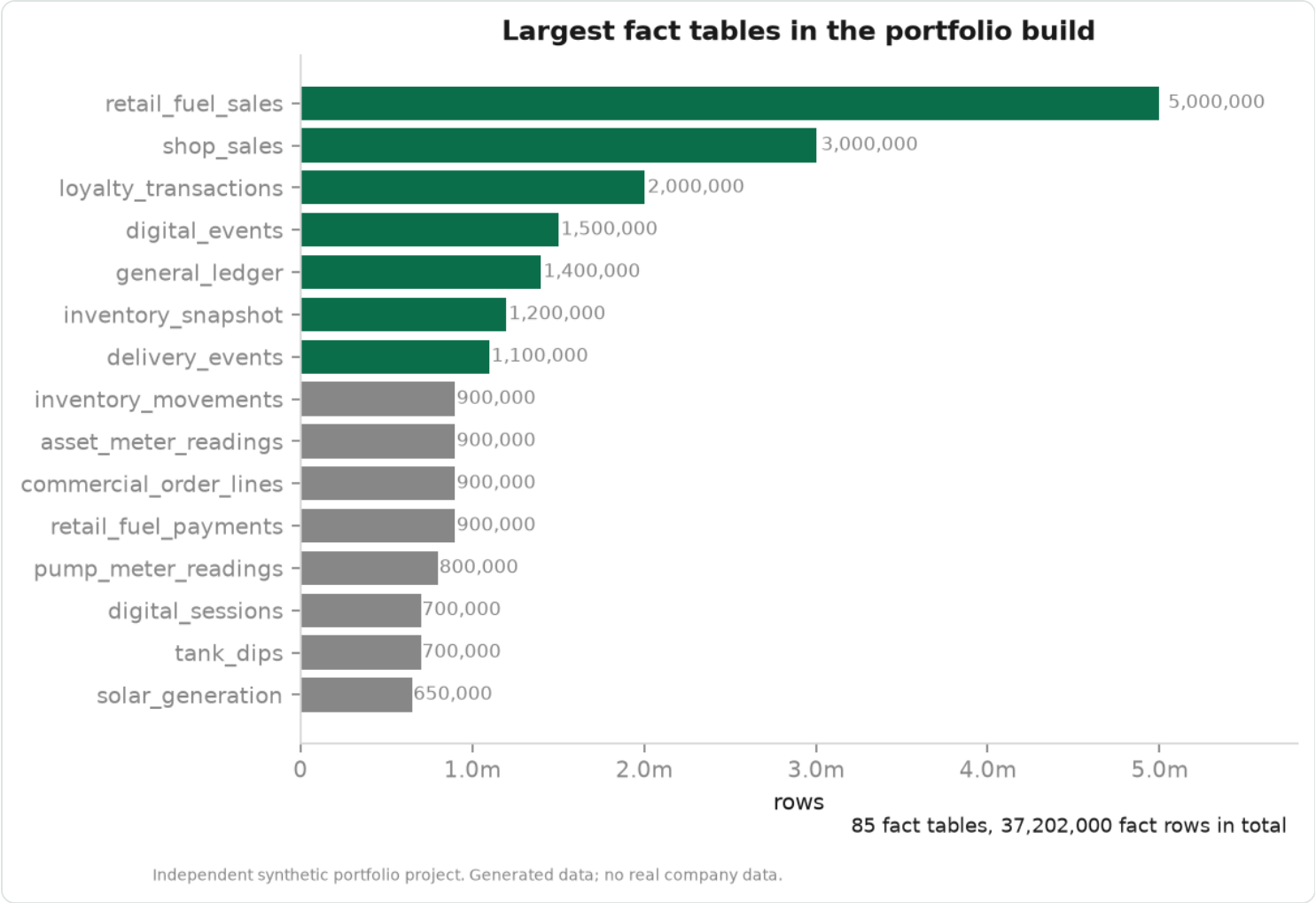
FACT TABLES	FACT ROWS	DEFECTS INJECTED	PROVINCES
85	37,202,000	62,924,526	9

The platform covers retail forecourt, commercial B2B, supply and distribution, LPG, lubricants, aviation, marine, loyalty, digital, EV and solar, asset maintenance, HSSEQ and finance — modelled on South Africa with all nine provinces and roughly 1,050 synthetic service stations. It is built to demonstrate the parts of data engineering that are hard, not the parts that demo well: a deliberately dirty landing zone, a cleansing layer whose output is measured rather than asserted, and models reported against baselines they have to beat.

Build

Table	Rows
fact_retail_fuel_sales	5,000,000
fact_shop_sales	3,000,000
fact_loyalty_transactions	2,000,000
fact_digital_events	1,500,000
fact_general_ledger	1,400,000
fact_inventory_snapshot	1,200,000
fact_delivery_events	1,100,000
fact_inventory_movements	900,000
fact_asset_meter_readings	900,000
fact_commercial_order_lines	900,000
fact_retail_fuel_payments	900,000
fact_pump_meter_readings	800,000
fact_digital_sessions	700,000
fact_tank_dips	700,000
fact_solar_generation	650,000
fact_customer_invoices	620,000
fact_site_energy_consumption	600,000
fact_customer_receipts	560,000
fact_commercial_orders	550,000
fact_mobile_payments	520,000
fact_site_daily_operations	500,000
fact_loyalty_points_balance	480,000

Table	Rows
fact_finance_revenue_cost	420,000
fact_forecourt_shift	420,000
fact_lpg_sales	400,000

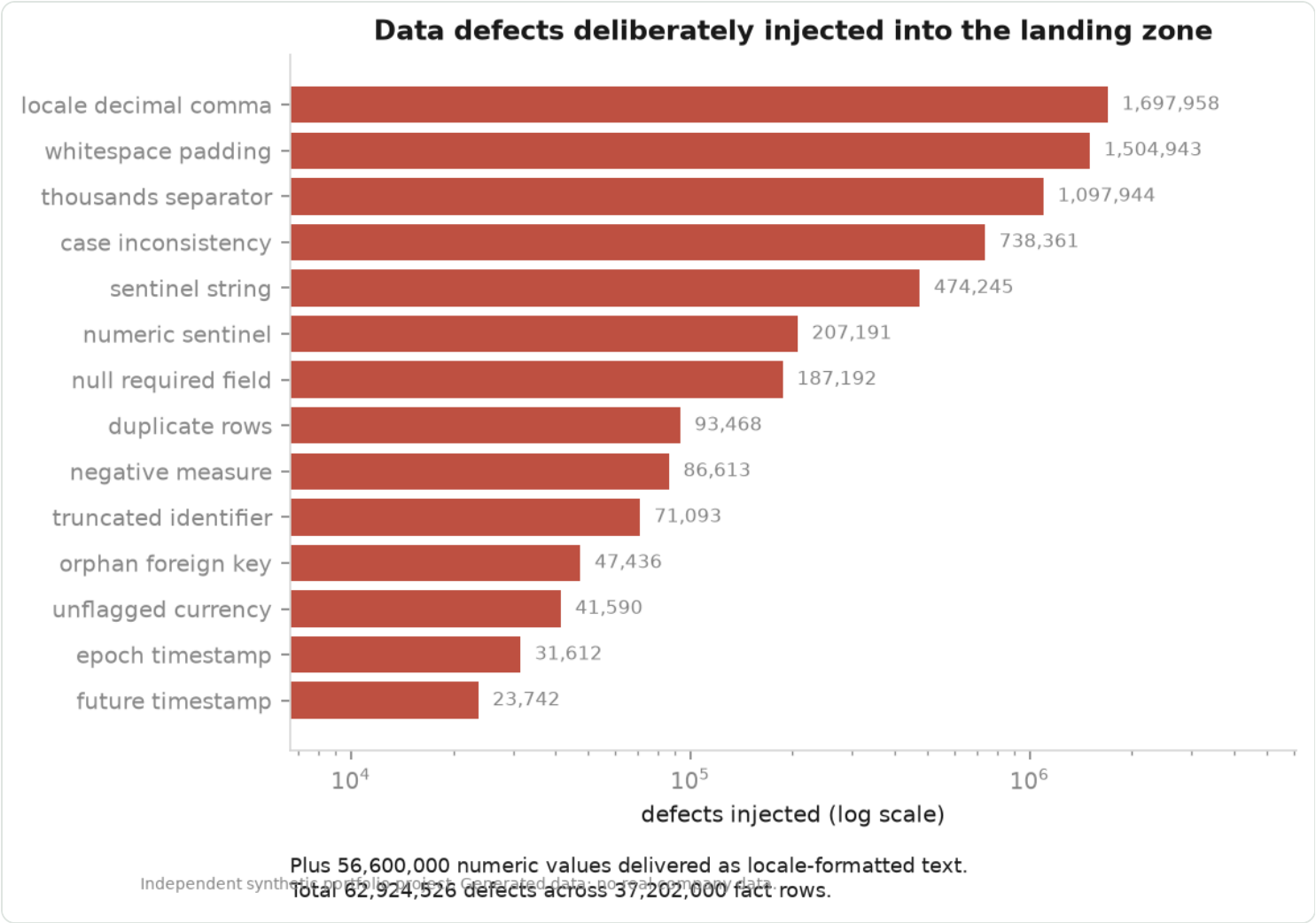


Largest fact tables in the portfolio build.

Data quality

The landing zone is dirty on purpose. A portfolio project built on perfect data demonstrates nothing, because the difficult part of the job is deciding what to do with the row where litres reads "1 234,5", the site code has a trailing space, and the same transaction arrived twice because the point-of-sale replayed its batch. Every defect is counted as it is injected, which turns cleansing from an assertion into a measurement.

Defect type	Defects
numeric delivered as text	56,600,000
locale decimal comma	1,697,958
whitespace padding	1,504,943
thousands separator	1,097,944
case inconsistency	738,361
sentinel string	474,245
numeric sentinel	207,191
null required field	187,192
duplicate rows	93,468
negative measure	86,613
truncated identifier	71,093
orphan foreign key	47,436
unflagged currency	41,590
epoch timestamp	31,612
future timestamp	23,742
derived value mismatch	12,256
unit of measure error	7,233
encoding mojibake	1,649

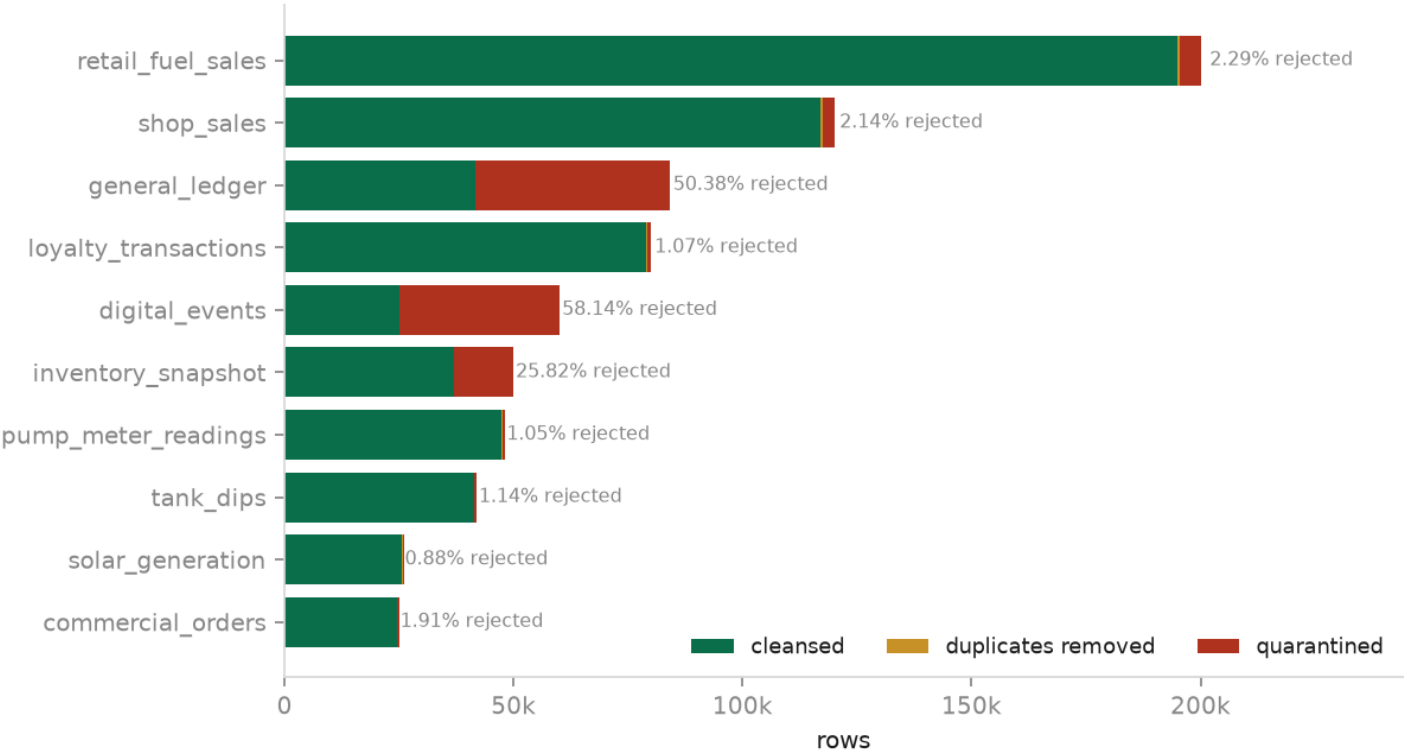


Defects deliberately injected into the landing zone.

Cleansing outcome

Not available in this run.

Every landed row is accounted for: cleansed, deduplicated or quarantined



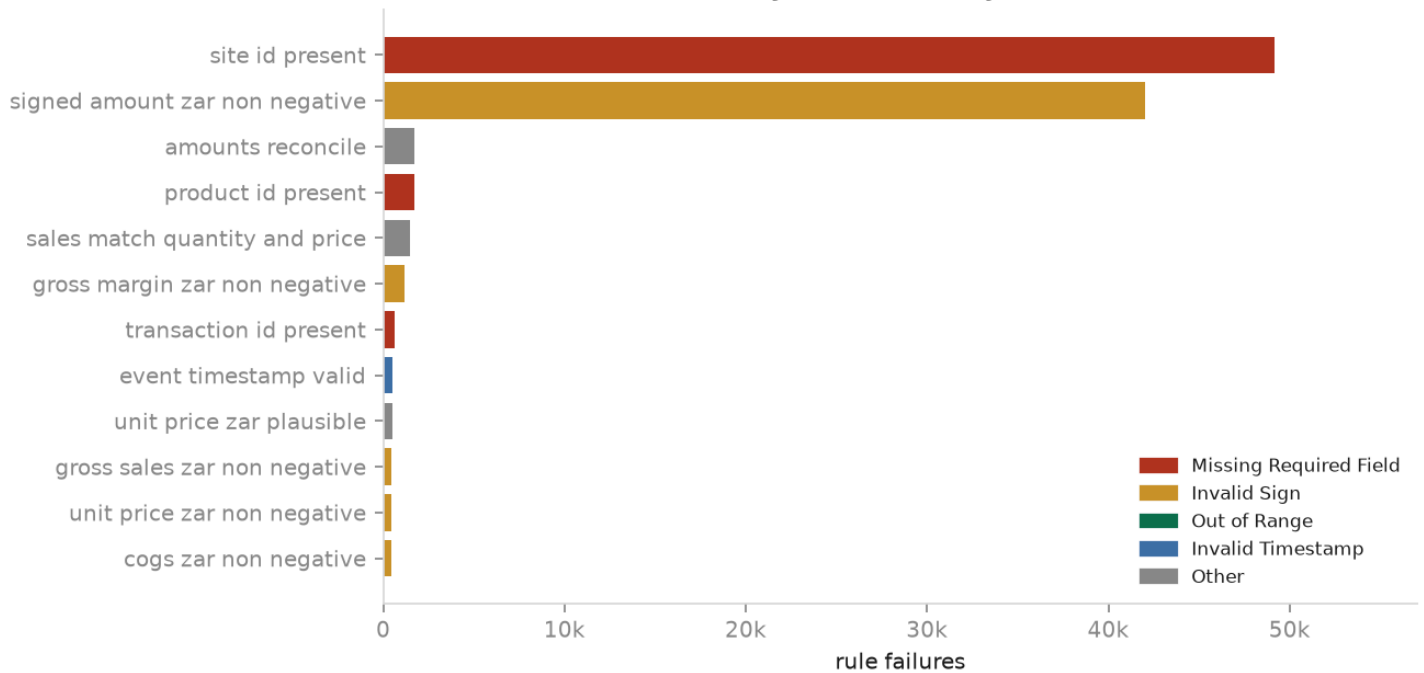
Drawn from drakens360_test_full.duckdb (194,934 retail transactions in gold). Independent synthetic portfolio project. Generated data; no real company data.

Every landed row is accounted for as cleansed, deduplicated or quarantined.

Why rows were rejected

Not available in this run.

Why rows were rejected



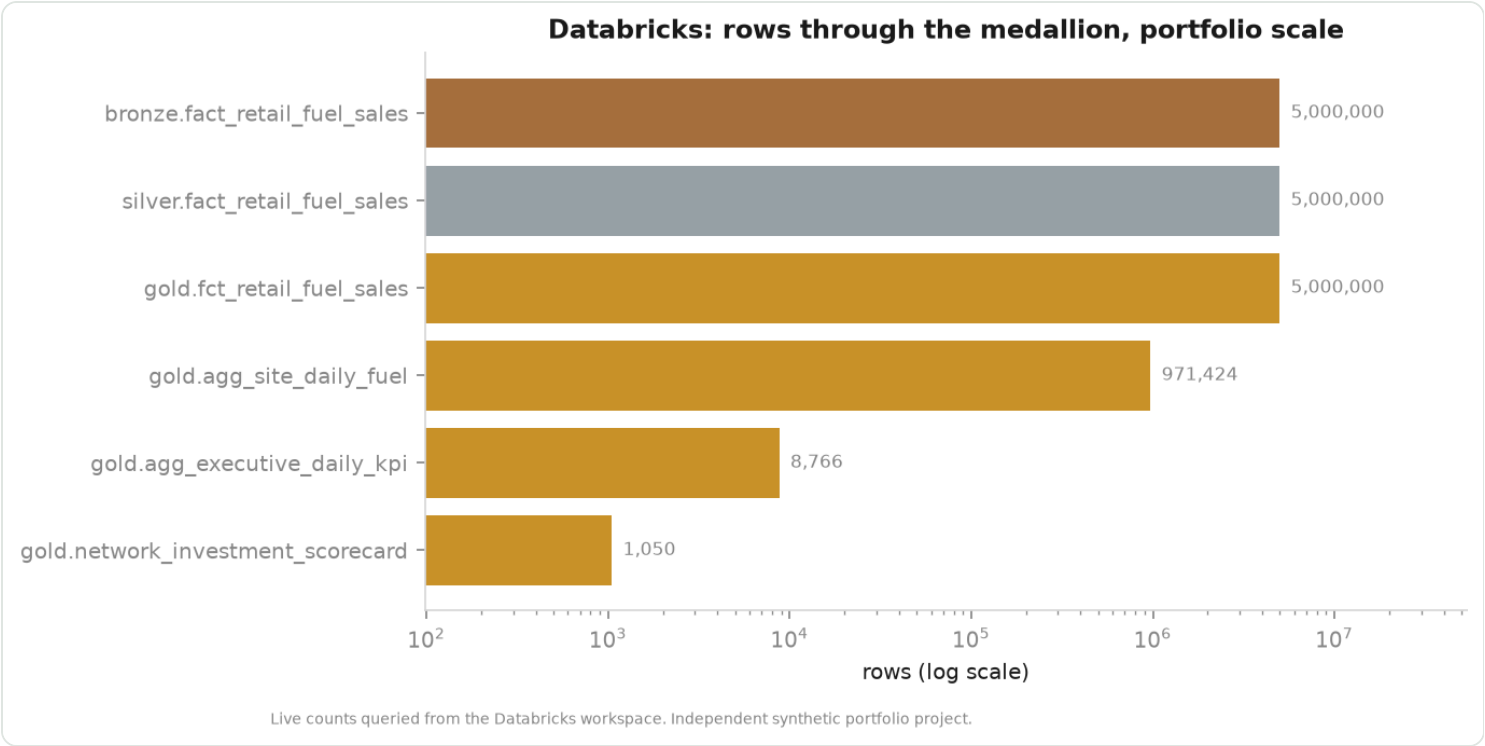
Drawn from drakens360_test_full.duckdb (194,934 retail transactions in gold). Independent synthetic portfolio project. Generated data; no real company data.

Rejection reasons. The operational response differs by rule.

Warehouse

Not available in this run.

A reconciliation control evaluates a published number as data rather than only as a test, so a control that fails names the figure that is wrong and by how much. Controls that fail are shown as failures; a report that quietly omits them would defeat the purpose of having them.



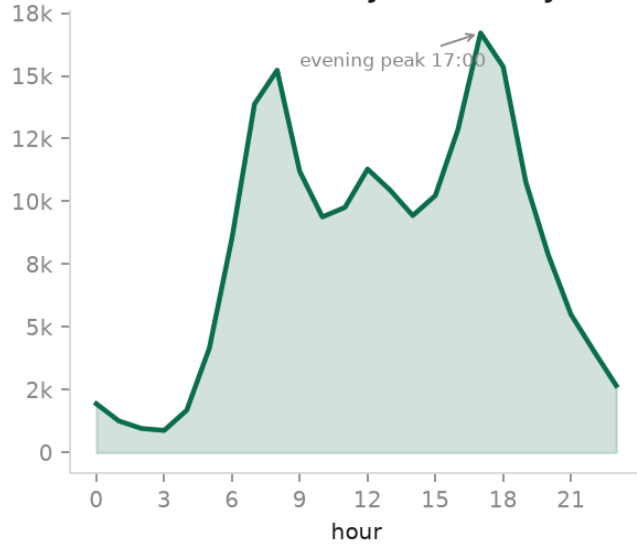
Live row counts through bronze, silver and gold on Databricks at portfolio scale.

Demand and trading rhythm

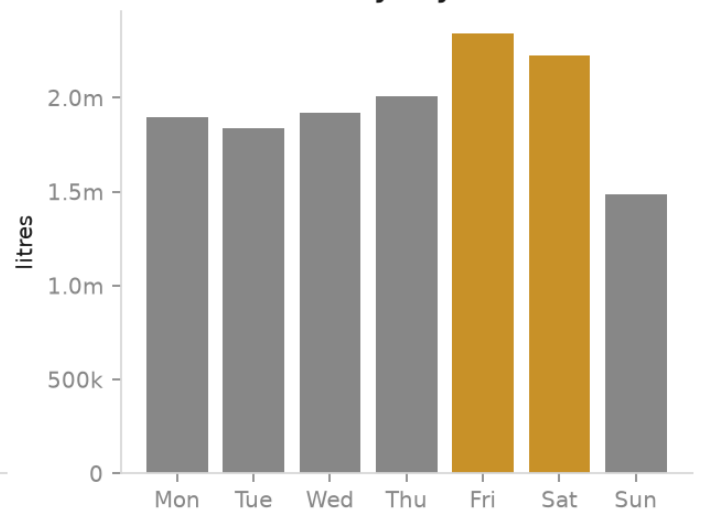
A forecasting model can only learn a pattern the data actually contains. These three figures are the evidence that the generator produces one: an hourly commute shape, a weekly rhythm, and a southern-hemisphere seasonal cycle that survives all the way through to the warehouse.

Generated demand follows real trading rhythms

Transactions by hour of day



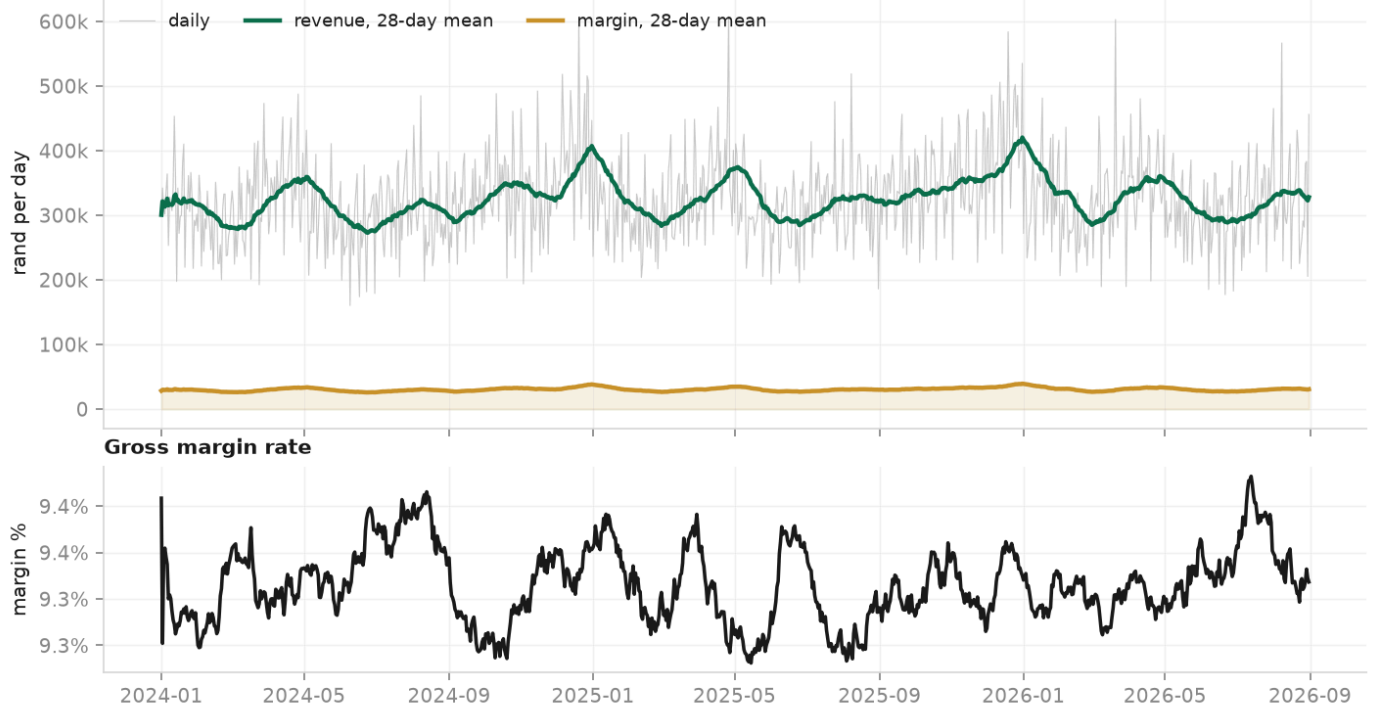
Volume by day of week



Drawn from drakens360_test.duckdb (196,057 retail transactions in gold). Independent synthetic portfolio project. Generated data; no real company data.

Commute peaks at 08:00 and 17:00, and a Friday-heavy week.

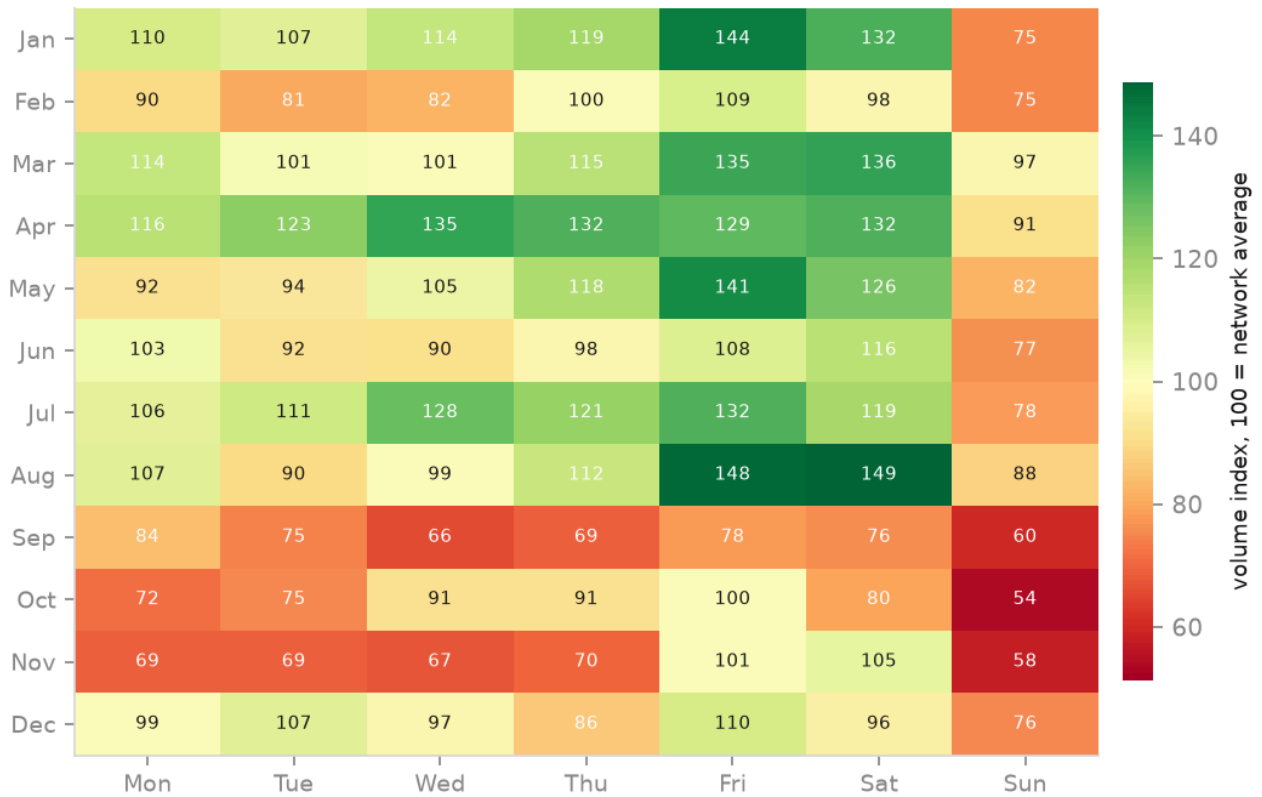
Network revenue and margin



Drawn from drakens360_test.duckdb. Independent synthetic portfolio project. Generated data; no real company data.

Revenue and margin across the modelled window, with the margin rate on its own axis. A rate and a total share an axis only in charts that say nothing about either.

Volume by month and day of week

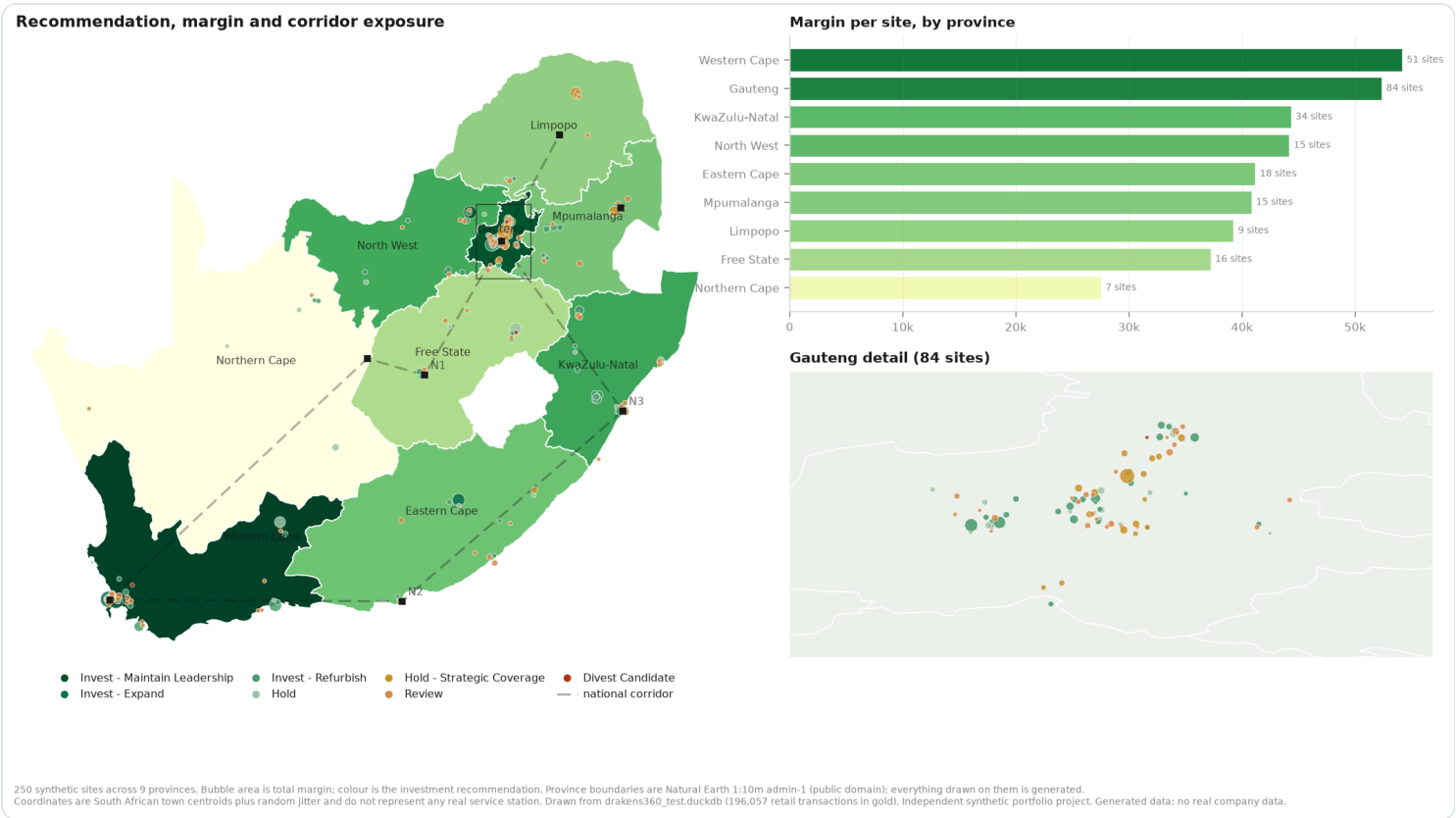


Southern-hemisphere seasons: December and January are the high-summer holiday peak, June and July the winter trough.

Drawn from drakens360_test.duckdb (196,057 retail transactions in gold). Independent synthetic portfolio project. Generated data; no real company data.

Month against day of week, indexed to the network average. December and January are the high-summer holiday peak; June and July the winter trough.

Network and investment



Provinces shaded by margin per site; bubble area is total margin and colour is the investment recommendation. The N1, N2 and N3 are drawn because corridor sites are protected from divestment regardless of score. Province boundaries are Natural Earth 1:10m admin-1, public domain; everything drawn on them is generated.

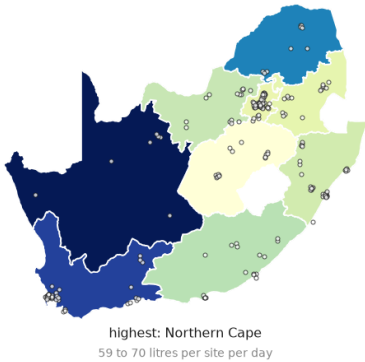
Measure	Value
Budget	R1.50bn
Funded projects	250
Capital committed	R1.384bn
Expected annual uplift	R353.1m
Portfolio ROI	25.5% per year
Blended payback	3.9 years
Provinces covered	9 of 9

Plan by province

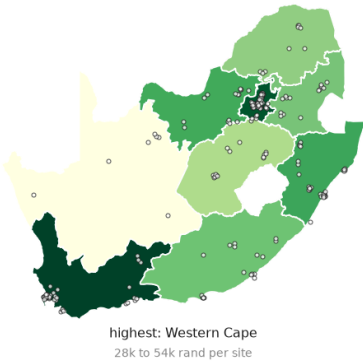
Province	Projects	Capex (R m)	Uplift (R m)
Gauteng	84	460.80	121.48
Western Cape	51	285.60	81.67
KwaZulu-Natal	34	190.80	42.80
Eastern Cape	18	97.70	22.64
Free State	16	87.00	18.99
North West	15	86.40	20.20
Mpumalanga	15	82.10	18.43
Limpopo	9	45.20	15.16
Northern Cape	7	42.20	10.72

The same network, read six ways

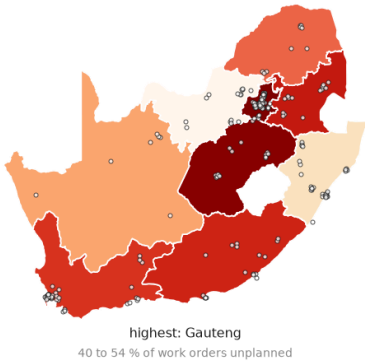
Throughput



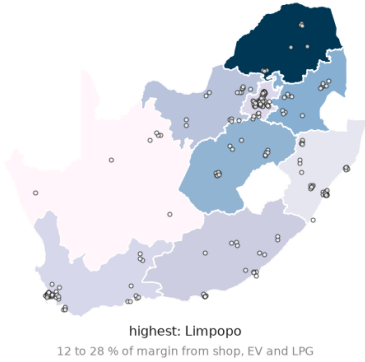
Margin per site



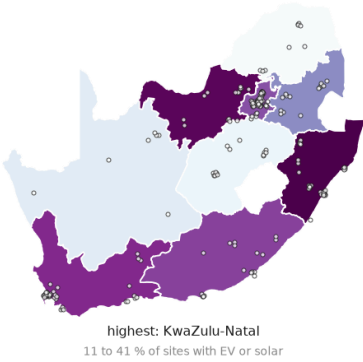
Asset condition



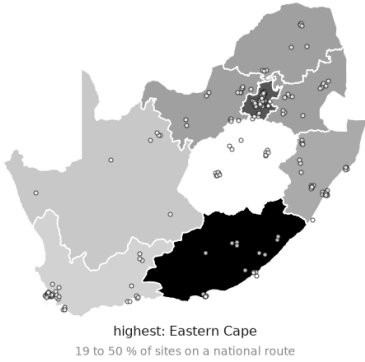
Non-fuel margin



New energy



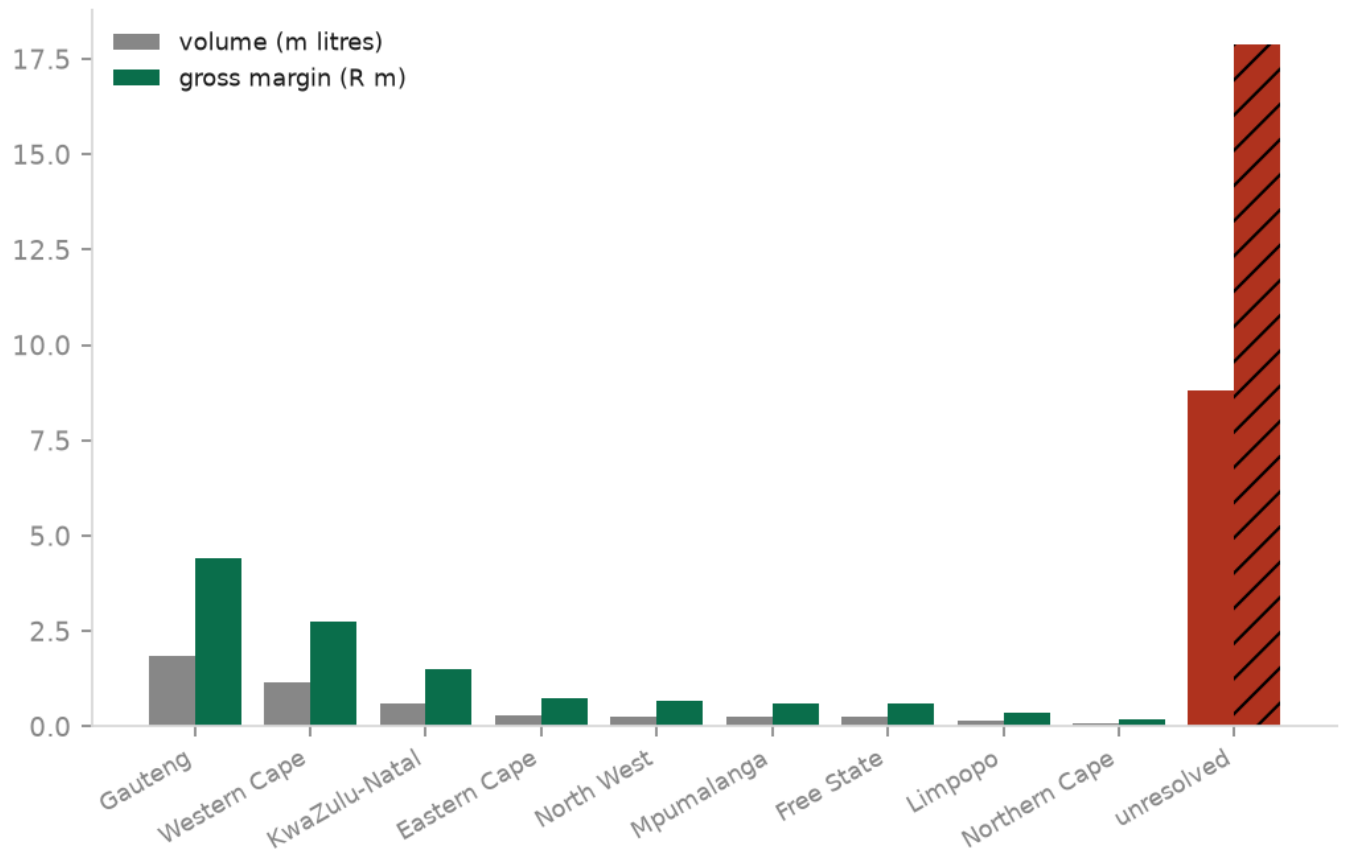
Corridor exposure



Provinces shaded by the panel's measure; dots are the synthetic sites. Identical in every panel so the six are comparable. Province boundaries are Natural Earth 1:10m admin-1 (public domain). Drawn from drakens360_test.duckdb (196,057 retail transactions in gold). Independent synthetic portfolio project. Generated data; no real company data.

The same network read six ways, on a fixed basemap and extent so the panels are directly comparable. A single map answers one question; a network planner asks six.

Volume and margin by province

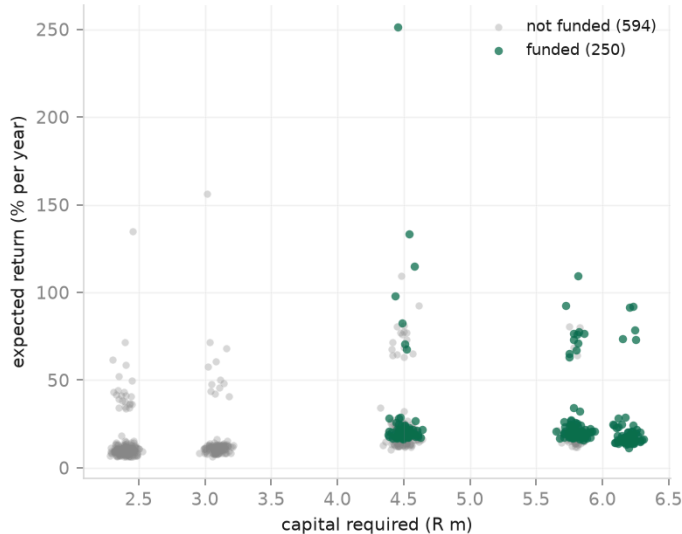


Drawn from drakens360_test.duckdb (196,057 retail transactions in gold). Independent synthetic portfolio project. Generated data; no real company data.

60% of margin sits on sites that did not resolve to a province. On a partial build this is expected -- the site dimension is incomplete -- and on a full build it would be a referential defect worth failing the run over.

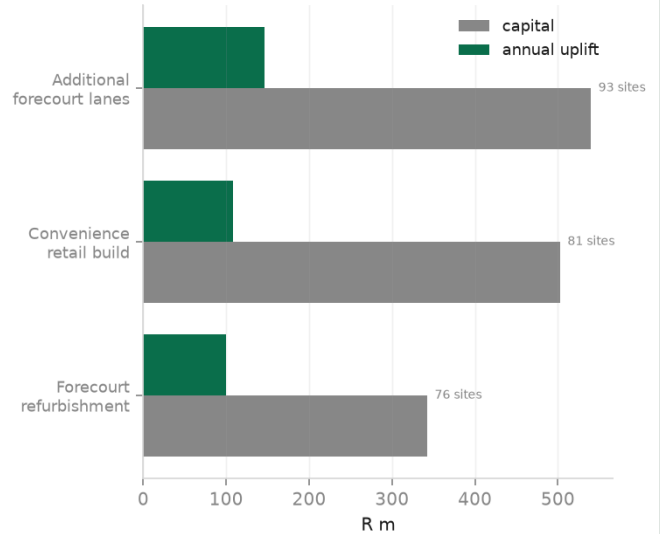
Volume and margin by province.

Candidate projects: return against capital

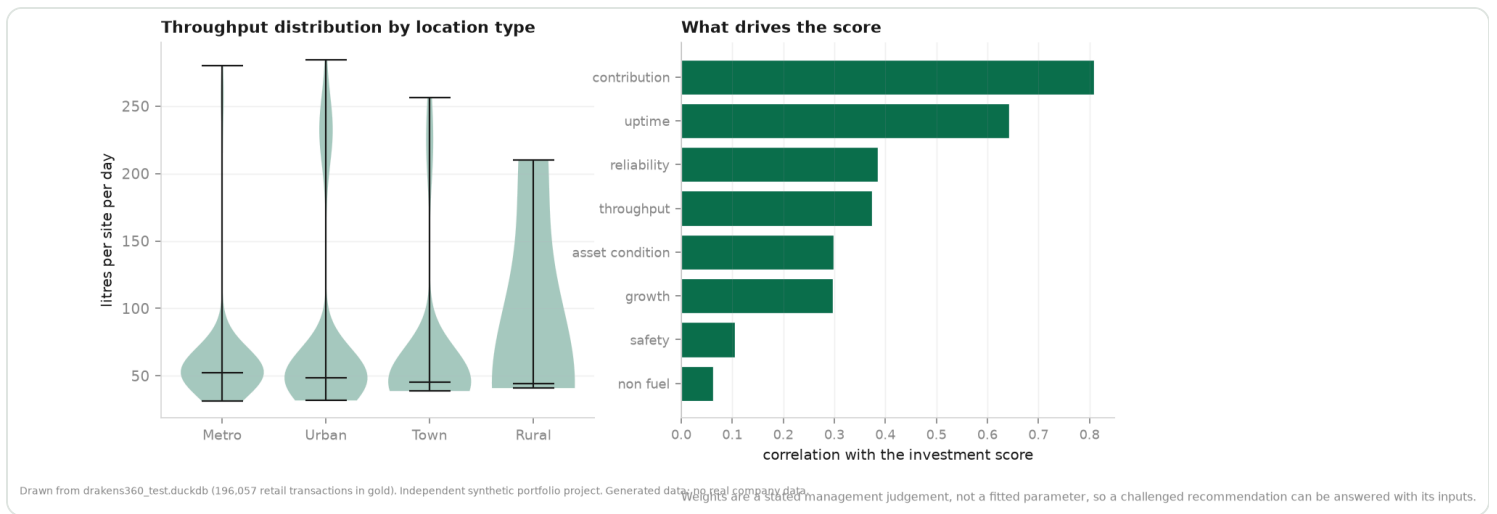


Drawn from drakens360_test.duckdb. Independent synthetic portfolio project. Generated data; no real company data.

Funded plan by intervention



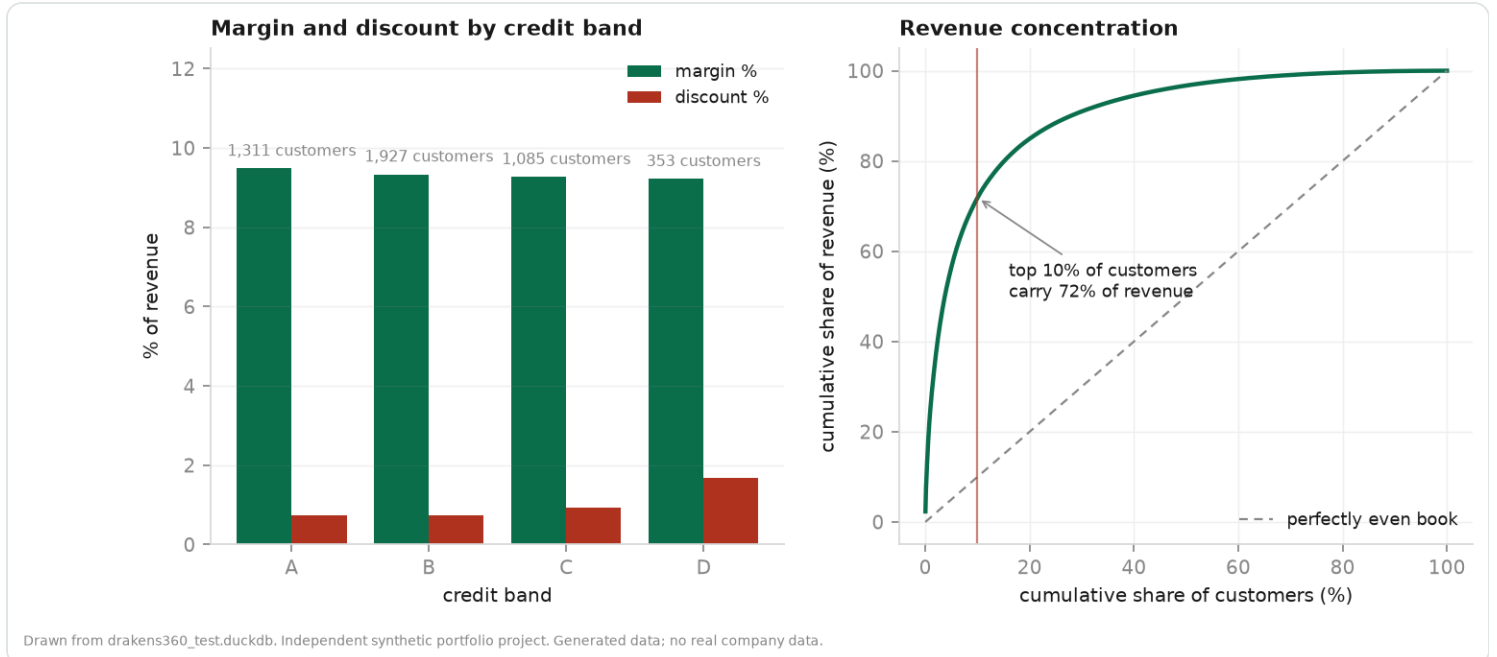
Every candidate project by return against capital, with the funded set highlighted. The funded set is not simply the top of the ROI ranking — provincial minimums and corridor protection pull specific projects in.



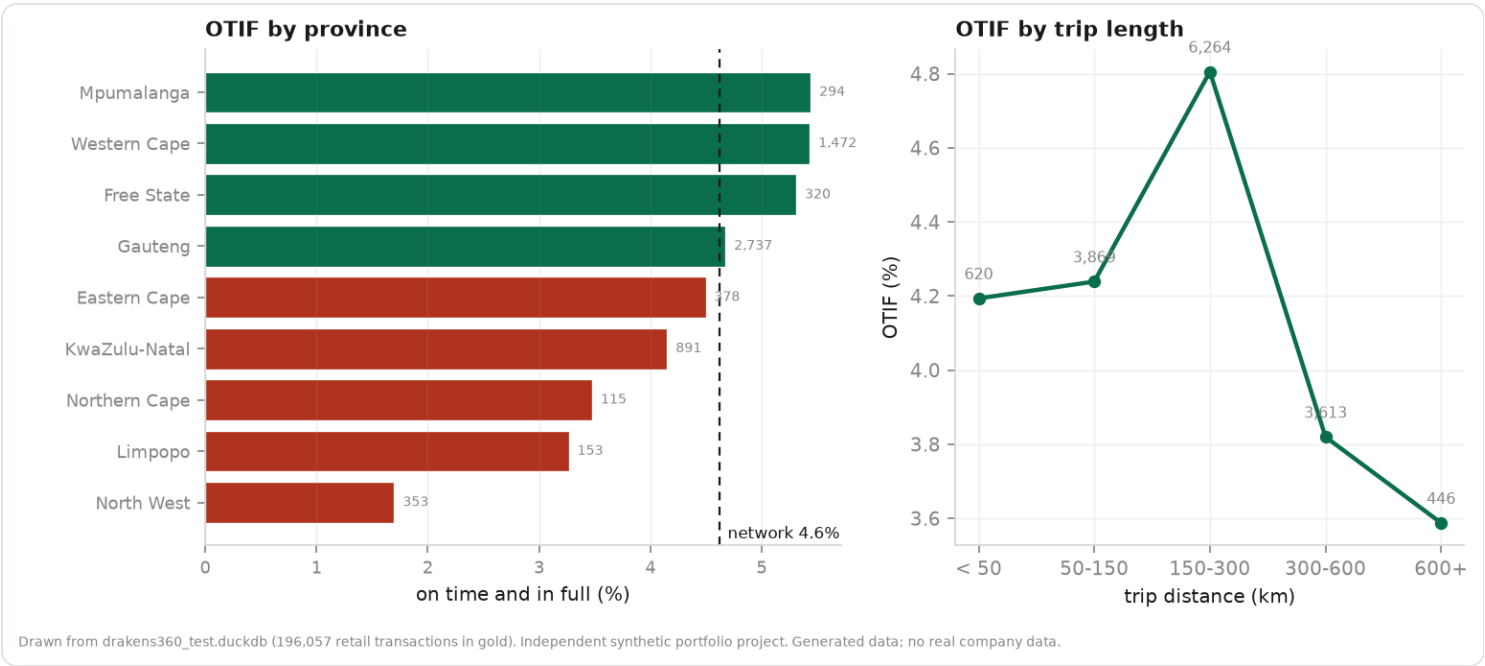
Left: throughput is skewed enough that a network average is a poor summary. Right: what actually drives the investment score, as the correlation between each component and the total.

Commercial and operations

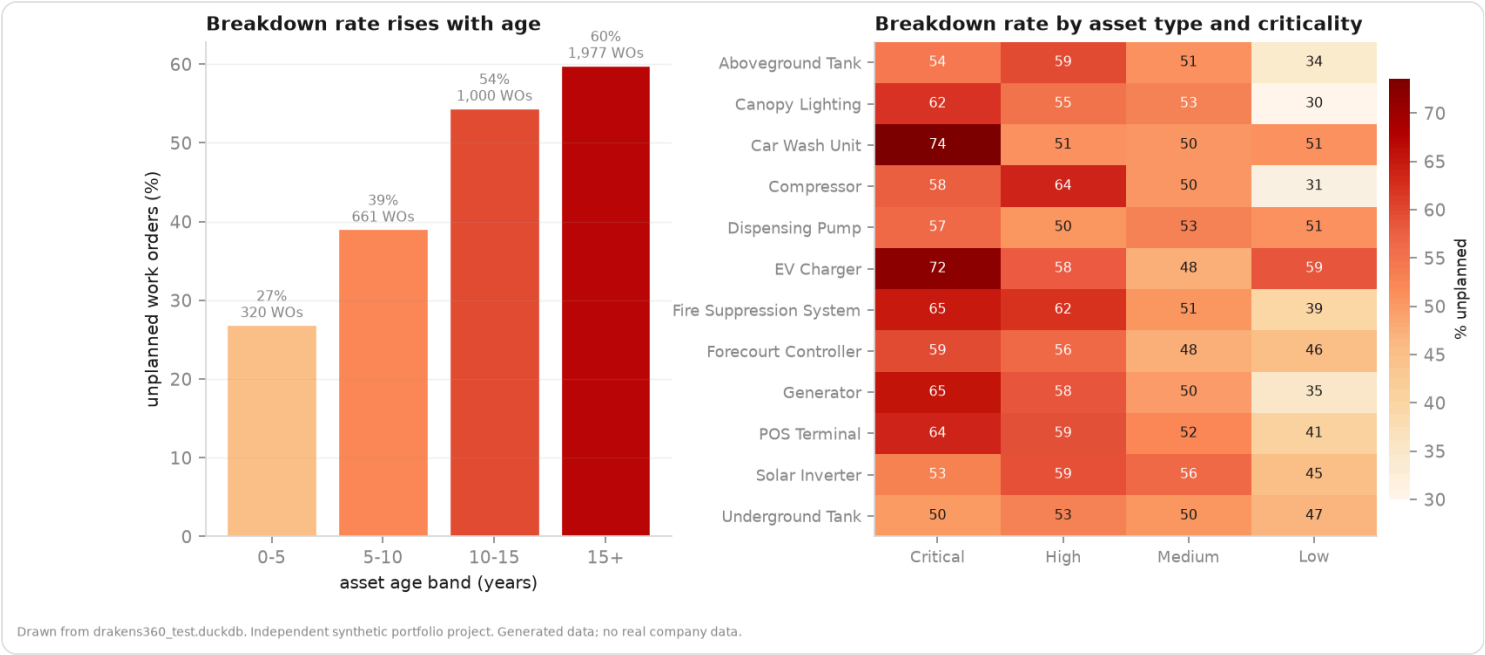
Four questions the business asks that the platform is built to answer, each computed once in gold rather than reassembled per report.



Left: whether the credit band means anything — worse-rated customers discount harder. Right: how concentrated the customer book is, which is a different risk from an even one.



OTIF broken out by province and by trip length. As a single network number it is the least useful form of it: broken out, it says whether lateness is a routing problem, a distance problem or one region’s problem.

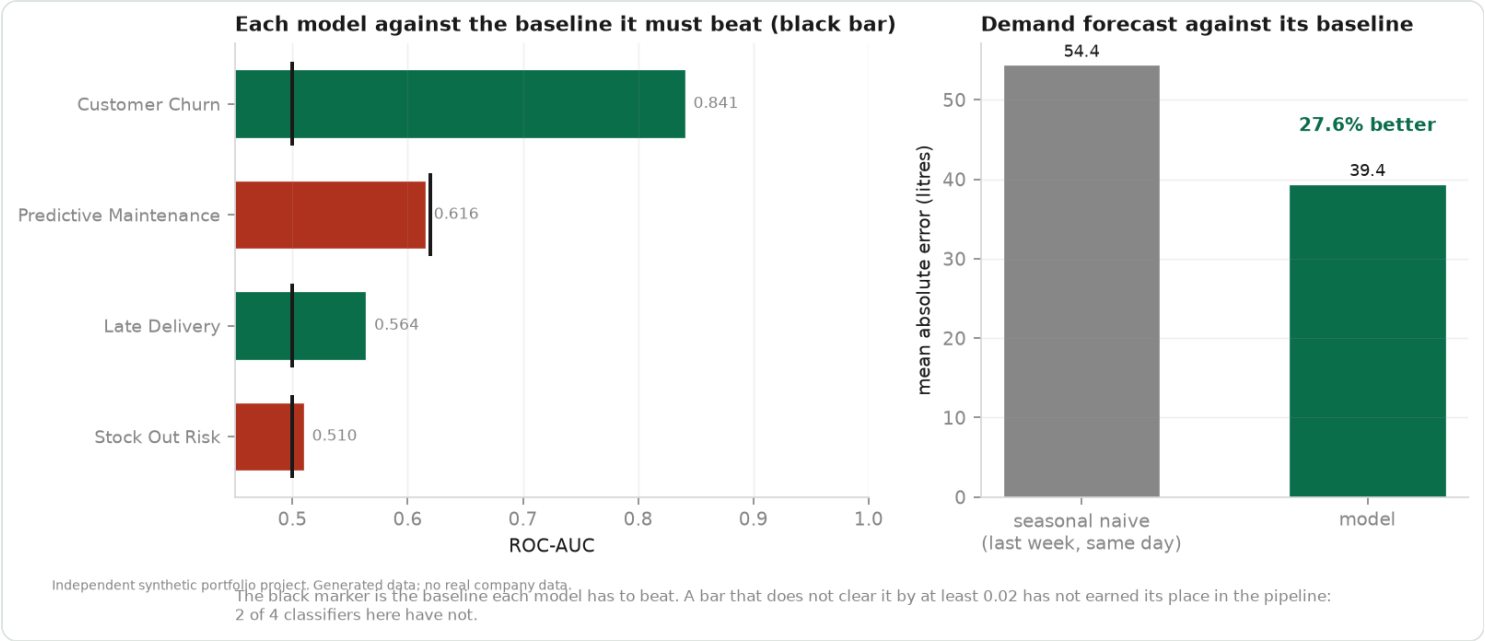


Breakdown rate against asset age and criticality. This is the signal the predictive-maintenance model is allowed to learn from, and the reason that experiment reports an age-only baseline.

Machine learning

Six experiments, each reported against a baseline it has to beat. Not all of them win, and all six are listed — a portfolio that shows only the models that worked is not showing the job.

Model	Headline result	Verdict
Customer Churn	ROC-AUC 0.841, 87% precision in the top decile	Usable — ranks well above chance
Demand Forecast	MAE 39.4 L against a seasonal-naive 54.4 L	Beats its baseline by 27.6%
Fraud Anomaly	1.0% of shifts flagged; cash share 29.4% against 23.9% for normal shifts	Flagged shifts differ from normal ones on both axes
Late Delivery	ROC-AUC 0.564, 1.76x lift in the top decile	Usable — the reviewed decile is enriched
Predictive Maintenance	ROC-AUC 0.616, 1.17x lift in the top decile, against a baseline of 0.620	Does NOT beat its baseline
Stock Out Risk	ROC-AUC 0.510	No usable signal



Each classifier against the baseline it has to beat, and the demand forecast against its seasonal-naive baseline. A bar that does not clear its black marker has not earned its place in the pipeline.

What the platform found in itself

Each of these was caught by the project’s own tests, observability or baselines, which is the point of having them.

A macro that silently corrupted every ratio

`safe_divide('a - b', 'c')` compiled to `a - (b / c)` because the macro did not parenthesise its arguments. Not an error — a plausible-looking wrong number. Caught by a range test asserting that `dim_product.base_margin_pct` falls between -5 and 100.

A general ledger that did not balance

Journal lines were posted independently, so debits exceeded credits by 46%. Caught by `obs_reconciliation_controls`, which evaluates double-entry as data rather than only as a test. Fixed by emitting journals as balanced pairs.

A partitioning choice that looked obvious and was wrong

The retail fact was partitioned on `date_key`, because every query filters on a date range. The maintenance job measured the result: 974 files averaging 0.2 MB, which OPTIMIZE can never merge because it does not cross partition boundaries. Under liquid clustering on `(date_key, site_id)` the same 5 million rows occupy 2 files averaging 94 MB — a 487-fold reduction in file count for identical data.

A leak that scored 0.998

The stock-out model was scored against the current inventory snapshot. Its target is `days_of_cover` below a threshold; `days_of_cover` is stock over usage; and both components were features. Excluding `days_of_cover`, which the code did with a comment explaining why, excluded nothing — the model divided one component by the other and reproduced the rule. Re-pointed at the next snapshot it scores 0.510.

A model with no baseline, losing to one variable

Predictive maintenance reported a 1.35x lift and passed review. It had no baseline. Ranking the queue by asset age alone scores ROC-AUC 0.620 against the model's 0.598, and the reported lift moves between 1.03 and 1.35 across capacity settings with no trend. Every experiment now computes and logs the baseline it has to beat.

Null foreign keys where an unknown member belonged

The landing zone contains referential drift by design — a site code in a fact that the dimension never received. Gold left-joined those facts and kept the null key, which drops the row from every inner join and from any total grouped by province: 5,070 retail transactions with real litres and real margin were invisible to regional reporting. They now resolve to a -1 unknown member, so nothing disappears from a total and the drift is countable.

A validity rule that deleted half the general ledger

The contract applied a blanket rule to every amount column: anything named *_zar must be non-negative. signed_amount_zar is negative on the credit side by design, so the rule quarantined 41,886 of the ledger's 84,000 lines — every credit leg of every journal. What survived was a ledger of debits with almost no credits, which reported a 99.98% imbalance that looked like a generator bug and was a contract bug. A validity rule inferred from a column name is a guess, and a wrong guess deletes data silently instead of failing loudly. Signed measures are now matched by naming convention and excluded, so a new net_ or _variance column is covered the day it is added.

Double-entry integrity is a property of the journal, not the line

Cleansing assesses rows one at a time, so when one leg of a balanced pair failed its contract the other leg survived alone. Each orphaned leg sat in the ledger as an unmatched debit or credit, and the ledger could not balance however correct the generator was — the imbalance was created by the cleansing step itself. The mart now publishes only journals whose lines still net to zero, and the orphans stay in quarantine beside the leg that was rejected. Debits and credits now agree exactly, to the cent.

A sentinel that passed every non-negativity check

The generator uses 0 as one of its numeric sentinels, and 0 satisfies every 'amount >= 0' rule. Stripping zero the way -999 is stripped is not an option, because zero litres and zero margin are legitimate readings. The contract now checks amounts against their own row instead: a transaction of 82 litres at R22.34 reporting R0.00 of sales is internally inconsistent whatever caused it.

Limitations

Stated plainly, because a report that claims completeness is misleading.

- The Databricks results are at portfolio scale (5,000,000 retail rows through bronze, silver and gold). The DuckDB warehouse figures in this report are at test scale, and each figure is stamped with the warehouse it came from. The two should not be read as one run.
- Predictive maintenance does not beat an age-only baseline, and stock-out risk has no signal at the current snapshot cadence. Both limits are in the generated data rather than in the estimators, and both fixes are changes to the generator: condition monitoring for the first, denser inventory readings for the second.
- The investment plan's rand figures rest on one stated benchmark — what a mid-sized site earns in fuel gross margin in a year — because the transaction fact is a sample and capital costs are not. Rank order and every ratio between sites are unaffected; only the level is rebased.
- Row filters are correct but their query cost under concurrent executive load has not been measured.

- There is no data subject access or erasure workflow. The masking and pseudonym design supports one; the process itself is not built.

Drakens Energy is a fictional company invented for this project. It does not exist. Every site, customer, employee, supplier, asset, coordinate, price, volume, margin and incident in this report is randomly generated. Site coordinates are South African town centroids plus random jitter and do not represent any real service station.